

Walchand College of Engineering, Sangli

Department of Computer Science and Engineering

Class: Final Year B.Tech(Computer Science and Engineering)

Year: 2025-26

Semester: 1

Course: High Performance Computing Lab

Practical No. 3

Exam Seat No: 22510019

Batch : B6

Title of practical:

Study and Implementation of schedule, nowait, reduction, ordered and collapse clauses

Problem Statement 1:

Analyse and implement a Parallel code for below program using OpenMP.

// C Program to find the minimum scalar product of two vectors (dot product)

Code snapshot :

```

5  int main() {
6      int i;
7      int n;
8
9      printf("Enter Size: ");
10     scanf("%d",&n);
11
12     int *a = malloc(n * sizeof(int));
13     int *b = malloc(n * sizeof(int));
14
15     long long dot = 0;
16
17     for (i = 0; i < n; i++) {
18         a[i] = i % 100;
19         b[i] = (n - i) % 100;
20     }
21
22     double start = omp_get_wtime();
23
24     #pragma omp parallel for reduction(+:dot_product)
25     for (i = 0; i < n; i++) {
26         dot += a[i] * b[i];
27     }
28
29     double end = omp_get_wtime();
30     printf("Minimum Scalar Product: %lld\n", dot);
31     printf("Time Taken: %f seconds\n", end - start);
32
33     free(a);
34     free(b);
35
36     return 0;
37 }

```

Screenshots:

```

soham@Vivobook16x-Soham: /mnt/d/SEM VII/HPCL/Assignment3$ ./ps1
Enter Size: 12
Minimum Scalar Product: 286
Time Taken: 0.004235 seconds

```

Information and analysis:

This program computes the scalar (dot) product of two integer vectors using OpenMP for parallelization. The vectors are initialized with sample values, and the dot product is calculated using a parallel for loop with reduction to efficiently sum the products across multiple threads.

For a vector size of **12**, the minimum scalar product was **286**, computed in **0.004235** seconds. While parallelization overhead is noticeable for small vectors, the approach scales well for larger data sizes, allowing significant speedup by utilizing multiple CPU cores.

Problem Statement 2:

Write OpenMP code for two 2D Matrix addition, vary the size of your matrices from 250, 500, 750, 1000, and 2000 and measure the runtime with one thread (Use functions in C to calculate the execution time or use GPROF)

- i. For each matrix size, change the number of threads from 2,4,8., and plot the speedup versus the number of threads.
- ii. Explain whether or not the scaling behaviour is as expected.

Screenshots:

```
soham@Vivobook16x-Soham:/mnt/d/SEM VII/HPCL$ cd Assignment3/
soham@Vivobook16x-Soham:/mnt/d/SEM VII/HPCL/Assignment3$ gcc -fopenmp ps2.c -o ps2
soham@Vivobook16x-Soham:/mnt/d/SEM VII/HPCL/Assignment3$ ./ps2
Size: 250, Time: 0.004915 seconds
Size: 250, Time: 0.004915 seconds
Size: 500, Time: 0.001922 seconds
Size: 500, Time: 0.001922 seconds
Size: 750, Time: 0.003290 seconds
Size: 750, Time: 0.003290 seconds
Size: 1000, Time: 0.006036 seconds
Size: 1000, Time: 0.006036 seconds
Size: 2000, Time: 0.020126 seconds
Size: 2000, Time: 0.020126 seconds
```

Information and analysis:

Problem Statement 3:

For 1D Vector (size=200) and scalar addition, Write a OpenMP code with the following: i. Use STATIC schedule and set the loop iteration chunk size to various sizes when changing the size of your matrix. Analyze the speedup. ii. Use DYNAMIC schedule and set the loop iteration chunk size to various sizes when changing the size of your matrix. Analyze the speedup. iii. Demonstrate the use of nowait clause.

Screenshots:

```
soham@Vivobook16x-Soham:/mnt/d/SEM VII/HPCL/Assignment3$ gcc -fopenmp ps3.c -o ps3
soham@Vivobook16x-Soham:/mnt/d/SEM VII/HPCL/Assignment3$ ./ps3
Enter Size: 6
STATIC Schedule with chunk 10 and nowait completed.
Time taken: 0.009280 seconds
```

Information and analysis:

This program performs scalar addition on a 1D vector using OpenMP with a static schedule, chunk size of 10, and the nowait clause. The loop is parallelized so that each thread processes a fixed chunk of elements, and the nowait clause allows threads to proceed without waiting for others to finish

Analysis:

For a vector size of 6, the operation completed in 0.009280 seconds. While the benefits of chunking and nowait are more evident with larger vectors, this setup demonstrates how scheduling and synchronization choices can impact parallel performance and efficiency, especially as data size increases.

Github Link: [🌐 HPCL/Assignment3 at main · Somzee5/HPCL](#)

Final Year B.Tech: High Performance Computing Lab 2025-26 Sem I